

C++ Programming: Problem Solving and Programming

Chapter 7 Template

Objective

- Introduction to Templates
- Why use Templates
- Basic Template functions
- Uses of Templates

Introduction

- Template in C++ is a feature to promote the writing of generic programs.
- A blueprint or formula for creating a generic class or function for reusability and flexibility. Instead of writing separate functions or classes for different data types.
 - A template lets us write a single, reusable function or class that works with any data type.
 - Useful when same operation is needed to be performed on different data types

Template – Why?

- Code reusability
 - Write only one function, use it with multiple data types
- Types safety
 - Ensure type correctness at compile time, reducing potential runtime errors.
- Reduced codes duplication
 - Avoid rewriting similar functions for each data type
- Ease of maintenance
 - Only one version of codes to update

Template – Syntax 1

- Basic syntax:-

```
template <typename T>  
T functionName(T parameter1, T parameter2)  
{  
    // function body  
}
```

Note:

T is a placeholder for a data type. You can use any name for the placeholder, but T (for Type) is a common convention. The `template <typename T>` declaration tells the compiler that the following function is a template and T is a generic type.

Template – Syntax 1 (Example 1)

- Example:-

```
template <typename T>
T add(T a, T b) {
    return a + b;
}
```

Output:

7

9.3

```
int main() {
    cout << add(3, 4) << endl;      // int
    cout << add(2.5, 6.8) << endl;  // double
    return 0;
}
```

Template – Syntax 1 (Example 2)

- Example:-

```
template <typename T>
T maximum(T a, T b) {
    return (a > b) ? a : b;
}
```

```
int main() {
    cout << maximum(3, 7) << endl;    // int
    cout << maximum(3.5, 2.1) << endl; // double
    cout << maximum('a', 'z') << endl; // char
    return 0;
}
```

Output:

7

3.5

z

Template – Syntax 2

- Multiple template parameter syntax
 - ✓ Templates take more than one parameter
 - ✓ Useful when a function works with different types

```
template <typename T1, typename T2>  
returnType functionName(T1 parameter1, T2  
parameter2)  
{  
    // function body  
}
```


Template – Syntax 2 (Example 1)

- Example:-

```
template <typename T1, typename T2>
void printPair(T1 a, T2 b) {
    cout << "First: " << a << endl;
    cout << "Second: " << b << endl;
}

int main() {
    cout << printPair(8, 42.5) << endl;
    cout << printPair("Hello", 2025) << endl;
    return 0;
}
```

Output:
First: 8
Second: 42.5
First: Hello
Second: 2025

Template – Syntax 2 (Example 2)

- Example:-

```
template <typename T1, typename T2>
bool compareValues(T1 a, T2 b) {
    return a == b;
}
```

```
int main() {
    int a = 69, b = 96;
    cout << "Are both values " << a << " & " << b;
    cout << " equal? " << compareValues(a, b);
    return 0;
}
```

Output:

**Are both values 69 & 96
equal? false**

Template – Exercise 1

Use template with single parameter:-

- Create a template function **squareValue** to square up a value.

The main function should prompt user for an integer value. Then display the square of the value after applying the template above.

Template – Exercise 2

Use template with multiple parameters:-

- Create a template function **swapValues** to swap values of two variables of different types. The main function should prompt user for an integer value and a float value. Then display the result of before and after the swapping.